

Week 6, Lecture 2: Live Coding a Power Analysis

Instructor notes. Part 1: prep reading. Part 2: podium cheat sheet

Before class:

- Print the WOD (`WOD_Week6_Lecture2_FindTheBugs`); bring its key.
- Open a blank R script and an AI assistant in a browser tab, both visible on the projector. Bump the font size in both.
- Have the Lab 4, Act 2 handout open for the checks in each part.
- Optional safety net: run the grid chunk below the night before and save it (`saveRDS(grid, "grid.rds")`) in case the live run is slow.

This file has two parts, both instructor-only. **Part 1** is read beforehand: the full working code with real output, the plan for which pieces to type by hand and which to hand to an AI, and a set of good and bad prompts to explore with the class. **Part 2** (after the page break, large print) is the podium copy.

Beat	Time
WOD: find the bugs	~5 min
Live coding: Lab 4, Act 2	~40 min
Discussion	~5 min

Source material: `labs/Lab4_Act2.Rmd` and its key (`labs/answer_keys/Lab4_Act2_Key.Rmd`). All code below matches the key, so the numbers agree with what groups should have found.

Where this sits in the week

- **Last week, Lab 4:** Act 1 validated the log-scale trend test. Act 2 (the monitoring design problem) never started in lab.
- **Monday:** WOD debrief, course check-in, and an overview of maximum likelihood.
- **Tuesday, Lab 5:** likelihood by hand (Act 1), then a first `optim()` fit (Act 2). Act 2 introduced the new AI policy: each coding question is labeled “Recommended: no AI” or “AI optional/encouraged,” students disclose their AI use after each one, and they turn in outputs, not code.
- **Today:** a step back to build Lab 4, Act 2 live. Two goals: get students’ coding going again after two math-heavy days, and show in practice what “build and test one function

at a time” means, with and without an AI assistant. The demo mirrors the new policy: the core functions by hand, the repetitive parts with AI.

Rationale for the flow

- **Type the first four functions by hand.** They are short, and each one has a check from the Lab 4 handout. Typing them live lets you say each check out loud before running it. This is the habit to model. WOD bug 1a is in the first of these functions, and two more bugs that didn’t make the WOD (listed below) fit naturally into the others.
- **Hand `estimate_power()` and the design grid to an AI.** This is where the code gets longer and more repetitive, which is where AI help is most useful. Try a bad prompt first, then a good one, and test whatever comes back against numbers you already know (the false-decline rate and the power at one design).
- **Don’t aim to finish the cost table live.** If time is short, show the precomputed grid and the cheapest designs, and point to the key’s recommendation. The point of today is process, not the final answer.

The WOD (~5 min)

Three minutes alone, two minutes with a neighbor on Part 2, then straight into the demo. The two fixes:

- 1a, `detect_prob()`: $1 - \text{pobs}^k$ should be $1 - (1 - \text{pobs})^k$. Check: `detect_prob(1)` should equal 0.02.
- 1b, `estimate_power()`: `detected <-` should be `detected[i] <-`. Check: the power should never come out exactly 0 or 1.

Both bugs run without an error and give plausible-looking wrong answers. Say so as you move into the demo: “it runs” isn’t a test, so every function today gets a check like these before we move on.

Two more bugs that didn’t make the WOD, to raise while typing the matching function in part A:

- `simulate_counts()`: `rbinom(1, ...)` instead of `rbinom(length(t_survey), ...)` returns one count, not 25.
- `detect_decline()`: `||` instead of `&&` counts any negative slope (or any significant increase) as a decline.

Live coding, part A: four functions by hand (~15 min)

Put subheadings in the script as you go (`# Detection probability ----` and so on), and say why: it makes the script easy to navigate, and it mirrors how the pieces were planned on

paper.

Detection probability. A bout of k person-days detects an animal unless every day misses it: $q(k) = 1 - (1 - p_{obs})^k$.

```
detect_prob <- function(k, pobs = 0.02) {  
  1 - (1 - pobs)^k  
}  
detect_prob(1)    # should equal pobs
```

```
## [1] 0.02
```

```
detect_prob(3)    # handout check: about 0.0588
```

```
## [1] 0.058808
```

Two checks: the handout's value, and the edge case $k = 1$, where the answer must equal p_{obs} . Mention that the edge case would have caught WOD bug 1a.

True abundance. $N_t = N_0 \lambda^t$, with $\lambda^{25} = 0.75$.

```
true_abundance <- function(t, N0 = 1000, lambda = 0.75^(1/25)) {  
  N0 * lambda^t  
}  
true_abundance(c(0, 25))    # should return 1000 and 750 together
```

```
## [1] 1000 750
```

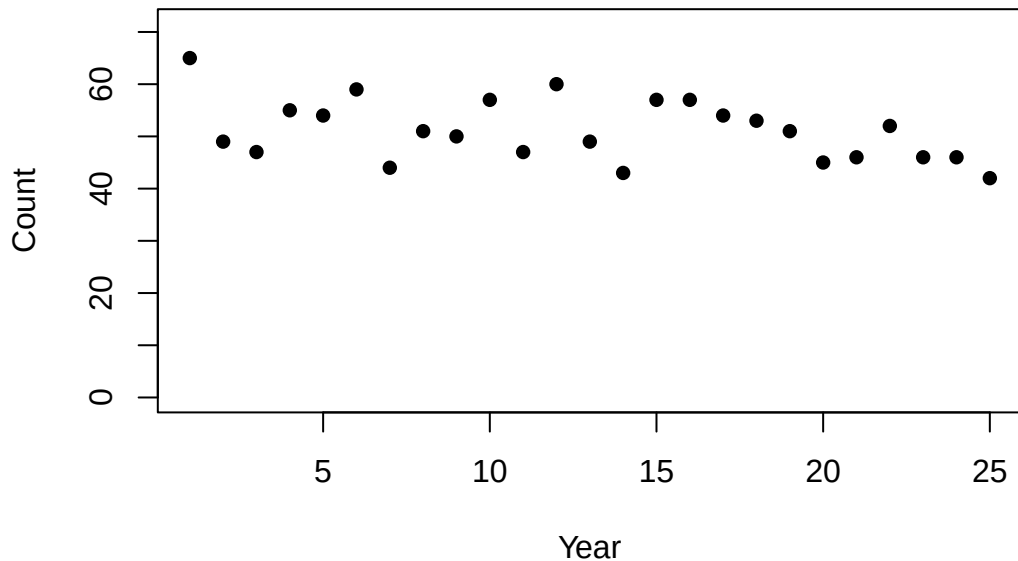
Test it with a *vector* of years, not one at a time. That is how the next function will call it.

Simulated counts. Each survey's count is $\text{Binomial}(N_t, q(k))$.

```
simulate_counts <- function(t_survey, k, N0 = 1000, lambda = 0.75^(1/25),  
                             pobs = 0.02) {  
  N_t <- true_abundance(t_survey, N0, lambda)  
  rbinom(length(t_survey), size = round(N_t), prob = detect_prob(k, pobs))  
}  
set.seed(11)  
counts <- simulate_counts(1:25, k = 3)  
length(counts)    # should be 25
```

```
## [1] 25
```

```
par(mar = c(4, 4.5, 0.5, 0.5))  
plot(1:25, counts, pch = 16, xlab = "Year", ylab = "Count",  
     ylim = c(0, max(counts) * 1.1))
```



Things to say out loud: `round()` because `size` must be a whole number; `length(t_survey)` draws, not 1 (the extra bug above); counts sit far below the true abundance (1000 falling to 750) because detection is under 6%. Plot it. A quick picture is a test too.

Decision rule. Negative slope and $p \leq \alpha$.

```
detect_decline <- function(t_survey, counts, alpha = 0.05) {
  s <- summary(lm(log(counts) ~ t_survey))$coefficients[2, ]
  unname(s[1] < 0 && s[4] <= alpha)
}
detect_decline(1:25, counts)
```

```
## [1] TRUE
```

Now a test with known truth: simulate a stable population ($\lambda = 1$) many times. The rule should fire only about 2.5% of the time, half of 5%, since significant *increases* don't count.

```
set.seed(12)
null_hits <- replicate(1000, {
  detect_decline(1:25, simulate_counts(1:25, 3, lambda = 1))
})
mean(null_hits)
```

```
## [1] 0.023
```

This is the check that would catch the || bug: with “or,” it would come out near 50%.

Live coding, part B: bring in the AI (~12 min)

Switch to the AI assistant for `estimate_power()`. Run the bad prompt first (prompt pair 1 or 2 below), ask the class what they expect to go wrong, then run the good prompt. Whatever

code comes back, test it against two numbers you already have:

1. **False-decline rate** at $\lambda = 1$, $k = 3$, $s = 1$: about 0.02 to 0.03 (just computed above).
2. **Power** at $k = 3$, $s = 1$: about 0.8 (from the Lab 4 key).

A reference version to compare against:

```
estimate_power <- function(k, s, R = 1000, lambda = 0.75^(1/25),
                           alpha = 0.05) {
  t_survey <- seq(1, 25, by = s)
  mean(replicate(R, {
    counts <- simulate_counts(t_survey, k, lambda = lambda)
    detect_decline(t_survey, counts, alpha)
  })))
}
set.seed(14)
estimate_power(3, 1, lambda = 1)    # false-decline rate

## [1] 0.025

estimate_power(3, 1)                # power

## [1] 0.822

run_time <- system.time(estimate_power(3, 1))["elapsed"]
run_time

## elapsed
##      0.3
```

Note the run time: about 0.3 s per design with $R = 1000$ on the machine that knit these notes, so the full 75-design grid takes roughly 22 s here, and likely longer on a classroom laptop. For the live demo, use $R = 200$ and say why: fast and rough first, then more replicates only where precision matters.

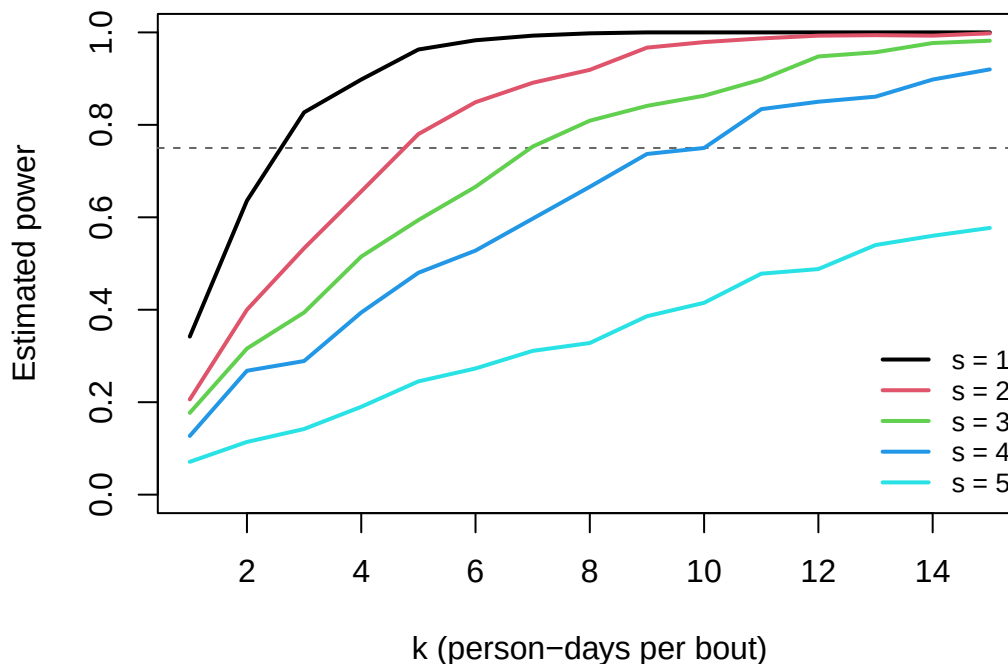
Things to look for in AI output (ask the class to spot them):

- Does it use the functions we already wrote, or rewrite everything from scratch (possibly with a different model)?
- Is the survey schedule right: `seq(1, 25, by = s)`, not `seq(1, 25, length.out = s)`?
- Does each replicate simulate *new* counts, or does it reuse one dataset?
- Does it store every replicate's result (WOD bug 1b)?
- Did it quietly add something we didn't ask for (a package, a different test, a `set.seed()` inside the function)?

Live coding, part C: the design space (~13 min)

```
set.seed(746)
grid <- expand.grid(k = 1:15, s = 1:5)
grid$power <- mapply(estimate_power, grid$k, grid$s,
                     MoreArgs = list(R = 1000))

par(mar = c(4, 4.5, 0.5, 0.5))
plot(NA, xlim = c(1, 15), ylim = c(0, 1), xlab = "k (person-days per bout)",
     ylab = "Estimated power")
abline(h = 0.75, lty = 2, col = "grey40")
for (ss in 1:5) {
  lines(grid$k[grid$s == ss], grid$power[grid$s == ss], col = ss, lwd = 2)
}
legend("bottomright", legend = paste("s =", 1:5), col = 1:5, lwd = 2,
      bty = "n", cex = 0.8)
```



Cost is \$200 per person-day plus \$2,000 per expedition:

```
cost_of <- function(k, s) length(seq(1, 25, by = s)) * (2000 + 200 * k)
grid$cost <- mapply(cost_of, grid$k, grid$s)
ok <- grid[grid$power >= 0.75, ]
head(ok[order(ok$cost), ], 5)
```

```
##      k s power  cost
## 55 10 4 0.750 28000
## 56 11 4 0.834 29400
```

```
## 37 7 3 0.753 30600
## 57 12 4 0.850 30800
## 58 13 4 0.861 32200
```

Points to make:

- Power rises with k and falls with s , and the two trade off at similar rates across most of the grid (compare with question 1d in the Lab 4, Act 2 handout).
- The cheapest designs sit right at 0.75. With 1,000 replicates, the Monte Carlo standard error near 0.75 is about $\sqrt{0.75 \times 0.25/1000} \approx 0.014$, so a design at 0.76 could easily be below the line. The key re-checks the top candidates with 5,000 replicates and recommends $k = 11$, $s = 4$ (\$29,400), which clears the bar with margin, over the cheapest $k = 10$, $s = 4$.
- If time allows, ask: would you trust the recommendation more if the AI had written the whole grid in one go? Why or why not?

Good and bad AI prompts to explore together

Each pair targets the same task. Show the bad prompt, ask what the class expects to go wrong, run it if time allows, then show the better version and ask what changed. You don't need to run all five: pairs 1, 2, and 4 fit best into part B.

Pair 1: the whole problem at once

- *Bad*: “Write R code for a power analysis for a declining toad population.”
- *Better*: “Write an R function `estimate_power(k, s, R)` that estimates the power of a monitoring design. It should call my existing functions `simulate_counts(t_survey, k)` and `detect_decline(t_survey, counts)`. Survey years are `seq(1, 25, by = s)`. For each of R replicates, simulate new counts and record whether a decline was detected. Return the fraction detected. Base R only.”
- *Why*: the bad prompt leaves the model unstated, so the AI fills it in with its own defaults (a canned t-test, a `pwr` package call, a Poisson or normal model, a different decision rule). The code may run and look professional while answering a different question.

Pair 2: the model in words vs. the model in equations

- *Bad*: “Simulate survey counts for a population that declines 25% over 25 years with imperfect detection.”
- *Better*: “Write `simulate_counts(t_survey, k, N0 = 1000, lambda = 0.75^(1/25), pobs = 0.02)`. True abundance is $N_t = N_0 \lambda^t$. Detection probability for a bout of k person-days is $1 - (1 - p_{obs})^k$. Each count is Binomial(N_t rounded, detection probability). Return one count per element of `t_survey`.”
- *Why*: “declines 25%” could mean linear or exponential decline; “imperfect detection” could be modeled many ways. If you’ve already worked out the equations on paper, give them to the AI.

Pair 3: debugging

- *Bad*: “Fix my code.” (pasting the whole script)
- *Better*: “This function should return 25 counts, one per survey year, but `length(simulate_counts(1:25, 3))` returns 1. Here is the function. Explain why it returns one value before suggesting a fix.”
- *Why*: the better prompt gives the symptom, the expected behavior, and a minimal example, and it asks for a diagnosis first. “Fix my code” invites the AI to rewrite everything, possibly introducing new bugs, and you learn nothing about what went wrong.

Pair 4: testing

- *Bad*: “Is this code correct?”
- *Better*: “Write three quick tests for `detect_decline()` with known answers: a clearly declining series should return `TRUE`, a flat series `FALSE`, and a clearly increasing series `FALSE`. Then tell me what fraction of `TRUE`s I should expect if I run it on many simulated datasets with no decline, and why.”
- *Why*: “Is this correct?” usually gets a confident yes plus a paraphrase of the code. Asking for tests with specific expected values turns the question into something you can check. Also check the tests themselves: an AI-written test sometimes compares the function to itself, which always passes.

Pair 5: leading questions

- *Bad*: “Show that survey frequency matters more than effort per bout.”
- *Better*: “Using `estimate_power()`, compute power for $k = 1, \dots, 15$ and $s = 1, \dots, 5$, and plot power against k with one line per s . Don’t interpret the results yet.”
- *Why*: a leading prompt invites the AI to find support for the conclusion you handed it. Ask for the computation, then interpret it yourself.

General guidance: AI for writing and testing code

These points are for the podium and for students. They restate what the lab handouts already ask for (plan on paper, one function at a time, record prompts) with reasons.

Before prompting

- **Do the thinking first.** Work out the model and the algorithm on paper (the pseudocode step in every lab). The AI is good at turning a clear plan into code, and poor at choosing the right model for your question.
- **Decide your tests before you see any code.** Know what numbers should come out (an edge case, a known-truth scenario, a value from the handout). Otherwise it’s easy to accept whatever looks reasonable.

While prompting

- **One function per prompt.** Give its name, arguments, what it returns, and the equations it implements.
- **Give the context.** Existing functions it should call, the data's structure, allowed packages (base R here).
- **Ask for explanations.** "Explain each line" and "What assumptions did you make?" often surface a substitution you didn't ask for.
- **Don't paste sensitive data** (unpublished data, locations of sensitive species, anything covered by a data agreement). Use a small made-up example with the same structure.

After the code comes back

- **Read it before running it.** Could you explain every line to a classmate? If not, ask, or look it up.
- **Run your tests.** Check shapes (`length()`, `dim()`), edge cases, and known-truth scenarios, not just "it runs without errors."
- **Debug with evidence.** Give the error message, a minimal example, and what you expected, and ask for a diagnosis before a fix.
- **Disclose it.** Note where you used AI and what you changed (the AI-use lines in Lab 5, Act 2 ask for this). Using AI never costs points in this course; hiding it does.
- **You own the result.** If AI-written code gives a wrong power estimate and the agency buys the wrong design, "the AI wrote it" is not an answer.

Discussion (~5 min)

Pick one:

- *Which of today's checks would you have skipped if you were in a hurry? What would that have cost you?*
- *Where did the AI help most today, and where would you not trust it without checking?*
- *Tuesday's Act 2 marked some coding tasks "Recommended: no AI." Which of today's tasks would you have labeled that way, and why?*

PART 2: PODIUM CHEAT SHEET

Terse on purpose. Full code, output, and rationale: Part 1.

Setup: R script + AI tab on projector, big fonts. `grid.rds` saved as backup.

WOD: find the bugs (~5 min)

- 3 min alone, 2 min neighbor
- 1a $1 - \text{pobs}^k \rightarrow 1 - (1 - \text{pobs})^k$; check `detect_prob(1) = 0.02`
- 1b `detected <-` $\rightarrow$ `detected[i] <-`; power never exactly 0 or 1
- Both silent: “it runs” isn’t a test

A. Four functions by hand (~15 min)

Function	Check (say it before running)
<code>detect_prob(k)</code>	<code>detect_prob(1) = 0.02</code> ; <code>detect_prob(3) $\approx$ 0.0588</code>
<code>true_abundance(t)</code>	<code>c(0, 25) $\rightarrow$ 1000, 750 together</code>
<code>simulate_counts(t, k)</code>	<code>length() = 25</code> ; plot it; counts $\ll N_t$
<code>detect_decline(t, y)</code>	$\lambda = 1$, many reps $\rightarrow$ TRUE $\approx$ 2.5%

- Subheadings as you go
- Extra bugs to raise: `rbinom(1, ...)` in `simulate_counts`; `||` in `detect_decline` ($\rightarrow$ ~50% under $\lambda = 1$)
- `round(N_t)`: size must be whole

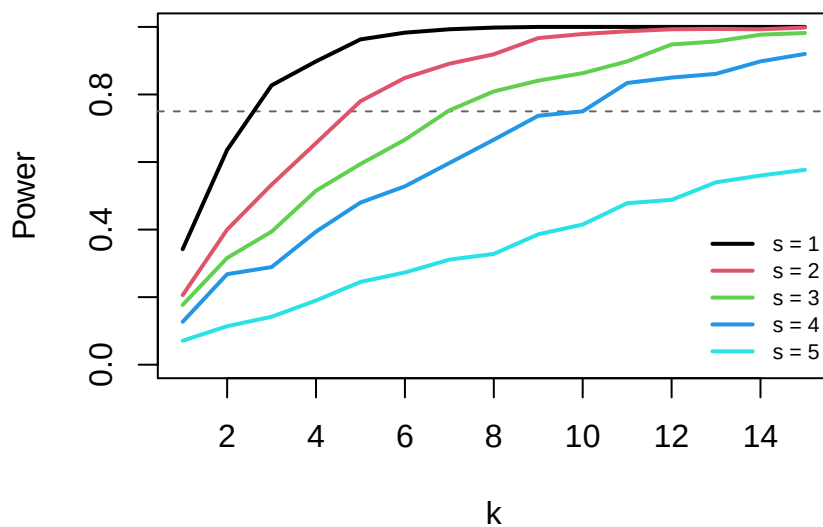
B. Bring in the AI (~12 min)

- `estimate_power(k, s, R)`: bad prompt first, class predicts problems, then good prompt
- Test against known numbers:
 - $\lambda = 1, k = 3, s = 1$: $\approx 0.02-0.03$
 - $k = 3, s = 1$: ≈ 0.8
- Look for: reuses our functions? `by = s`? new counts each rep? stores every rep? added anything unasked?
- `system.time()` $\rightarrow$ use `R = 200` live

Prompt pairs (use 1, 2, 4):

1. “Power analysis for a declining toad” vs. named function, calls our functions, `by = s`, base R
2. “Declines 25%, imperfect detection” vs. the equations
3. “Fix my code” vs. symptom + expected + minimal example + diagnose first
4. “Is this correct?” vs. three tests with known answers
5. “Show that s matters more” vs. compute and plot, don’t interpret

C. Design space (~ 13 min)



- Cost = bouts $\times$ (\$2,000 + \$200 k)
- k and s trade off at similar rates
- Cheapest designs sit on 0.75; MC SE $\approx$ 0.014 at $R = 1000$
- Key's pick: $k = 11$, $s = 4$, \$29,400 (margin over cheapest)

AI guidance (weave in throughout)

- Think first: model + pseudocode on paper
- Decide tests before seeing code
- One function per prompt: name, args, return, equations
- Give context: existing functions, data, base R
- Ask “explain each line” / “what did you assume?”
- No sensitive data; use a toy example
- Read before running; run your tests
- Debug with evidence; diagnose before fixing
- Disclose AI use; it never costs points, hiding it does
- You own the result

Discussion (~5 min)

- Which check would you skip in a hurry? Cost?
- Where did AI help most? Where not trust it?
- Which of today’s tasks would you label “Recommended: no AI”? Why?